SLAMFormer-∞: Infinite SLAM Transformer for Unbounded Frontend and Backend Processing

Zhijian Fang*, Weicheng Zheng*, Yijun Yuan*†, Weibang Wang, Zhuoguang Chen,
Chang Sun, Junhao Huang, Kenan Li, Minghui Qin, Hang Zhao†
IIIS, Tsinghua University
https://tsinghua-mars-lab.github.io/SLAMFormer-Infinity
{yuanyj, hangzhao}@mail.tsinghua.edu.cn

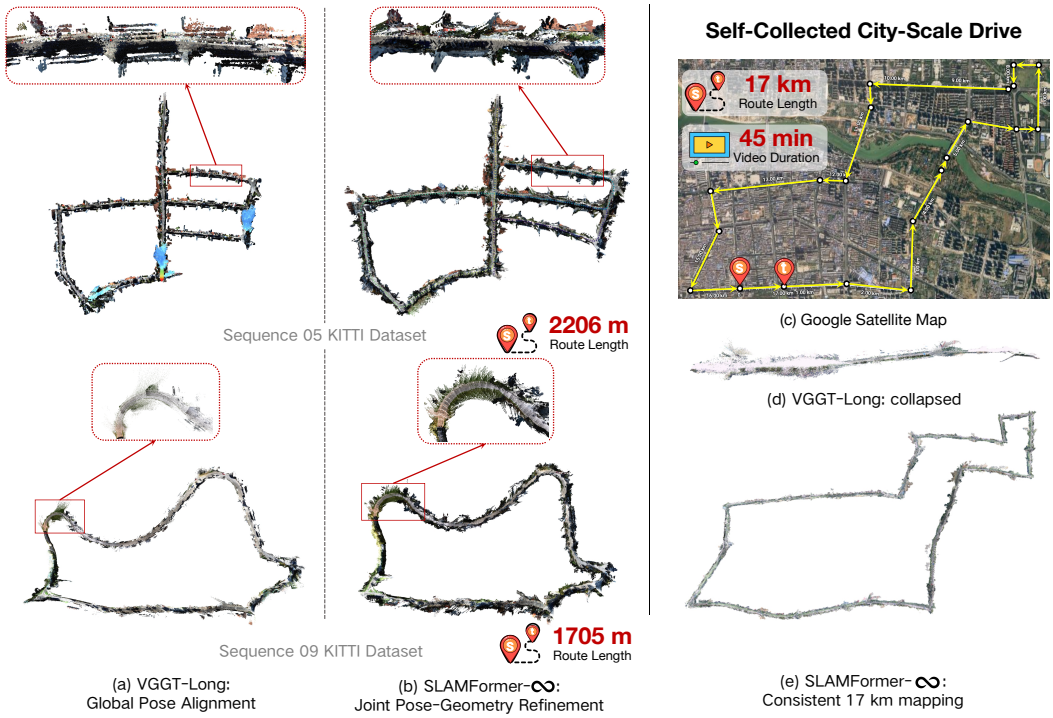

Figure 1: **City-scale reconstruction visualization.** VGGT-Long performs global pose alignment but leaves local geometry largely unrefined, while SLAMFormer-∞ jointly optimizes pose and dense geometry. On a self-collected 17 km urban drive, SLAMFormer-∞ further maintains a consistent large-scale map where VGGT-Long collapses.

Abstract: We introduce the Infinite SLAM Transformer (SLAMFormer-∞), the first geometric transformer capable of supporting both long-range frontend and backend processing without an explicit distance bound. Instead of relying on a first-frame-anchored formulation, SLAMFormer-∞ employs memory conditions to define flexible coordinate systems and scales for input frames, enabling more expressive structural conditioning. Built upon this formulation, the frontend preserves efficient local computation, while the backend jointly optimizes long-range trajectories and scene geometry in a globally consistent manner. Experimental results demonstrate that SLAMFormer-∞ achieves superior or highly competitive performance in both trajectory estimation and scene reconstruction across large-scale datasets. Notably, SLAMFormer-∞ generalizes to extremely long trajectories, successfully operating on sequences exceeding 17km.

Keywords: Dense Mono SLAM, Long-range Reconstruction

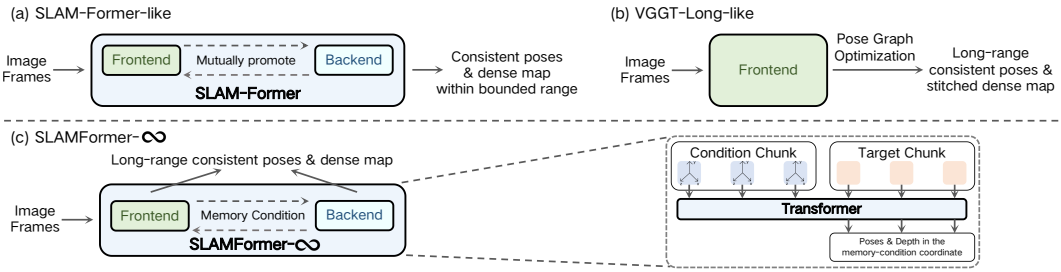

Figure 2: **SLAM Transformer Comparison.** (a) Single-model SLAM-Former for global consistent pose and map. (b) VGGT-Long with optimized long-range poses and stitched maps. (c) Ours retains the single-model while obtains long-range global consistent pose and map with a conditional design.

1 Introduction

Simultaneous Localization and Mapping (SLAM) plays a fundamental role in autonomous systems by enabling robot to localize itself and construct maps of previously unknown environments. Among different SLAM paradigms, monocular SLAM is particularly important for its simplicity and low hardware cost, requiring only a single RGB camera for autonomous perception and navigation.

Early monocular SLAM systems [1, 2], primarily focused on camera trajectory estimation with sparse feature-based maps. Although effective for localization, these approaches captured only limited scene geometry. To recover dense three-dimensional structure, later methods introduced dense monocular SLAM through dense bundle adjustment [3] or learning-based depth prediction [4].

More recently, neural scene representations and geometric foundation models have significantly advanced monocular SLAM. Gaussian Splatting-based methods [5, 6] demonstrate that neural rendering representations enable high-quality dense reconstruction from monocular input. Meanwhile, Geometric foundation-based methods [7, 8] leverage geometric transformers to predict camera pose, and scene geometry from multi-view observations, followed by backend pose-optimization for global consistency. Taking one step further, SLAM-Former [9] introduces transformer-based global attention for trajectory-geometry refinement without loop detection and optimization. Together, these methods move dense monocular SLAM toward learned, globally consistent, and geometry-aware systems.

However, recent transformer-based monocular SLAM systems suffer from two fundamental limitations. On one hand, fully data-driven approaches such as SLAM-Former [9] rely entirely on learned trajectory modeling, making their long-range performance bounded by the scale and trajectory distribution of training data. On the other hand, systems such as MASt3R-SLAM [7] and VGGT-SLAM [8] address long-range consistency through pose-centric optimization, where the backend primarily refines camera poses while leaving scene geometry largely fixed. As a result, trajectory correction and geometric reconstruction remain decoupled rather than being jointly optimized within a unified framework.

To address these limitations, we propose SLAMFormer-∞, an infinite SLAM Transformer for unbounded frontend and backend optimization as in Fig. 2. Unlike previous transformer-based SLAM systems, our method preserves memory condition throughout transformer inference, enabling them to act as persistent geometric anchors. Based on this design, SLAMFormer-∞ supports an efficient local frontend for long-range tracking, while enabling iterative backend that jointly optimizes both trajectory and geometry over arbitrarily long sequences.

The contributions of this paper are summarized as follows:

- We propose a novel Infinite SLAM Transformer that supports an unbounded long-range frontend tracking and backend optimization within a conditioned transformer framework.
- We introduce Pose-Geometry Graph Optimization (PGGO) with transformer that jointly refines long-range trajectory and scene geometry in a globally consistent manner.

- Experimental results demonstrate significantly improved reconstruction quality while achieving highly competitive localization performance against state-of-the-art methods.

2 Related Work

2.1 Geometric Transformer

Recent geometric foundation models have reshaped multi-view 3D perception by replacing hand-crafted correspondence and optimization with transformer-based geometry regression [10, 11]. DUSt3R [10] predicts dense pointmaps from image pairs, turning matching and triangulation into learned geometric regression, while MASt3R [11] further strengthens pairwise 3D matching. Beyond pairwise inference, Fast3R [12], VGGT [13], and Pi3 [14] extend this paradigm to feed-forward multi-view reconstruction, with VGGT jointly predicting cameras, depth, and pointmaps from multiple views. However, these models are still mainly formulated for bounded input sets, where all views fit into a finite attention context and the learned coordinate behavior is tied to training-time sequence ranges.

Recent long-range geometric transformers relax this assumption through memories, recurrence, test-time adaptation, or chunked processing [15, 16, 17]. For example, TTT3R [15] adapts model states during inference for recurrent reconstruction, while extend to long videos through local reconstruction and pose stitching. However, their long-range consistency still mainly relies on state management, coordinate resets, pose propagation, or submap alignment, rather than unified transformer refinement over distant poses and geometry.

SLAMFormer-∞ takes coordinate-conditioning view of long-sequence geometric reasoning: fixed condition chunks define the local reference geometry for active-frame prediction, so the transformer can operate on a bounded context while preserving long-range pose and geometry information.

2.2 Learning-based SLAM

Learning-based SLAM has gradually moved from hand-crafted visual frontends to neural geometric modules. Optical flow-based systems such as DROID-SLAM [3] and SceneFactory [18] construct dense structure through dense pixel matchings, while neural implicit and Gaussian-splatting SLAM methods optimize dense maps through rendering objectives, with MonoGS [5] demonstrating real-time monocular SLAM with 3D Gaussian Splatting. These methods still rely on explicit optimization or iterative rendering-based updates to maintain trajectory and geometry consistency.

More recent methods use geometric foundation models as feed-forward reconstruction priors. MASt3R-SLAM [7] builds dense SLAM from MASt3R matching and pointmap prediction, VGGT-SLAM [8] and VGGT-Long [19] construct and globally align local VGGT submaps, and SLAM3R [20] follows a local-clip reconstruction and registration strategy. These methods provide strong local geometry, but global consistency is still mainly imposed by external registration, submap alignment, pose-graph optimization, or fusion.

SLAM-Former [9] further moves toward transformer-native SLAM by integrating frontend tracking, mapping, and backend refinement into a single model. However, its global refinement relies on accumulated trajectory representations and historical KV states, tying long-range inference to a growing sequence-level state. SLAMFormer-∞ follows this unified-transformer direction, but replaces growing-state refinement with pose-geometry graph optimization (PGGO), enabling iterative joint refinement of camera poses and dense geometry with the same transformer.

3 Methodology

3.1 Problem Formulation

Given a streaming monocular image sequence $\mathcal{I}_{1:N} = \{\mathbf{I}_1, \ldots, \mathbf{I}_N\}$, the goal of SLAM is to estimate in real time both the camera trajectory $\mathcal{X}_{1:N} = \{\mathbf{g}_1, \ldots, \mathbf{g}_N\}, \quad \mathbf{g}_n \in SE(3)$, and the scene geometry representation $\mathcal{P}_{1:N} = \{\mathbf{P}_1, \ldots, \mathbf{P}_N\}$, while maintaining both accurate incremental tracking and global geometric consistency.

SLAM-Former [9] formulates SLAM as inference within a single transformer model: $p(\mathcal{X}, \mathcal{P} \mid \mathcal{I})$, replacing modulation with end-to-end prediction over trajectories and scene structure.

Specifically, it consists of a frontend and backend:

$$\text{Frontend:} \quad \mathcal{M}_n = f_\theta^f(\mathcal{I}_n, \mathcal{M}_{1:n-1}), \tag{1}$$

$$\text{Backend:} \quad \hat{\mathcal{M}}_{1:n} = f_\theta^b(\mathcal{M}_{1:n}). \tag{2}$$

where $\mathcal{M}$ denotes map token representations, and poses and geometry are decoded via a head function: $(\mathcal{X}, \mathcal{P}) = f_\psi(\mathcal{M})$.

The frontend operates causally for incremental tracking, while the backend performs global refinement over the full history.

3.2 SLAMFormer-∞

However, this formulation is fundamentally constrained by the distribution of training trajectories, preventing effective generalization in long-range sequences.

To address this limitation, we introduce Infinite SLAM transformer (SLAMFormer-∞), modeling: $p(\mathcal{X}, \mathcal{P} \mid \mathcal{I}, \mathcal{I}_C, \mathcal{X}_C)$, where the memory condition $(\mathcal{I}_C, \mathcal{X}_C)$ defines a reference coordinate system.

A key distinction from SLAM-Former is that ours performs both frontend and backend inference in a local coordinate system defined by the condition, rather than in a global coordinate system.

Given a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ over keyframes, SLAMFormer-∞ operates on local neighborhoods ($\mathcal{N}$):

$$\text{Conditional Frontend:} \quad \mathcal{M}_n = f_\theta^f(\mathcal{I}_n, \mathcal{I}_{n-k:n-1}, \mathcal{C}_{j \in \mathcal{N}_{(n-k)}}), \tag{3}$$

$$\text{Conditional Backend:} \quad \hat{\mathcal{M}}_{n-w:n} = f_\theta^b(\mathcal{I}_{n-w:n}, \mathcal{C}_{j \in \mathcal{N}_{(n-w)}}). \tag{4}$$

where $\mathcal{C}_\cdot$ denotes neighboring conditioning context $(\mathcal{I}_\cdot, \mathcal{X}_\cdot)$, $w \in \mathcal{W}(n)$ is the nearest anchor retriever out of window set $\mathcal{W}(n)$. Note that in frontend, $\mathcal{I}_{n-k:n-1}$ and $\mathcal{C}$ assists $\mathcal{I}_n$ with only KV caches obtained from previous backend and frontend processing.

However, this relative formulation prevents direct global full-attention inference as in SLAM-Former [9], motivating an iterative backend for global consistency.

3.3 Pose-Geometry Graph Optimization (PGGO)

Our backend jointly processes poses and geometries. We formulate the task as a Pose-Geometry Graph Optimization (PGGO) that jointly refines trajectory and scene geometry under long-range relational constraints. Specifically, PGGO operates over a pose-geometry interaction graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where node set is defined as $\mathcal{V} = \mathcal{X} \cup \mathcal{P}$ with $\mathcal{X}$ and $\mathcal{P}$ denoting pose nodes and geometry nodes. The edges is defined as $\mathcal{E} = \mathcal{E}_\mathcal{X} \cup \mathcal{E}_\mathcal{P}$, where $\mathcal{E}_\mathcal{X}$ connects poses explicitly with relative pose constraints, while $\mathcal{E}_\mathcal{P}$ implicitly captures geometric correlation through transformer attention f_θ.

For each frame (n), we define the joint state variable as: $\mathbf{x}_n = (\mathbf{g}_n, \mathbf{P}_n)$. Given an initialization $\tilde{\mathbf{x}}_n^0 = (\tilde{\mathbf{g}}_n^0, \tilde{\mathbf{P}}_n)$, obtained from pose graph optimization or frontend prediction, our goal is to jointly refine poses and geometries for global consistency:

$$\mathbf{x}^* = \arg\min_{\{\mathbf{x}_n\}} \sum_n \left\| \mathbf{x}_n - f_\psi \circ f_\theta(\mathcal{I}_n, \mathcal{C}_{n \in \mathcal{N}_{n-w}}) \right\|^2. \tag{5}$$

To solve above optimization, for $\mathbf{x}_n \in \mathcal{V}$, we use an iterative neural refinement:

$$\hat{\mathbf{x}}_n^{k+1} = f_\psi \circ f_\theta(\mathcal{I}_n, \mathcal{C}_{n \in \mathcal{N}_{n-w}}),$$

followed by direct pointmap updates together with dampled pose updates:

$$\tilde{\mathbf{P}}_n \leftarrow \hat{\mathbf{P}}_n^{k+1}, \quad \tilde{\mathbf{g}}_n^{k+1} = \exp[(1-\alpha)\log(\tilde{\mathbf{g}}_n^k) + \alpha \log(\hat{\mathbf{g}}_n^{k+1})], \tag{6}$$

with α the damping factor. We write $\tilde{x}_n^{k+1} = h_\theta(\tilde{x}_n^k)$ for simplicity of PGGO iteration.

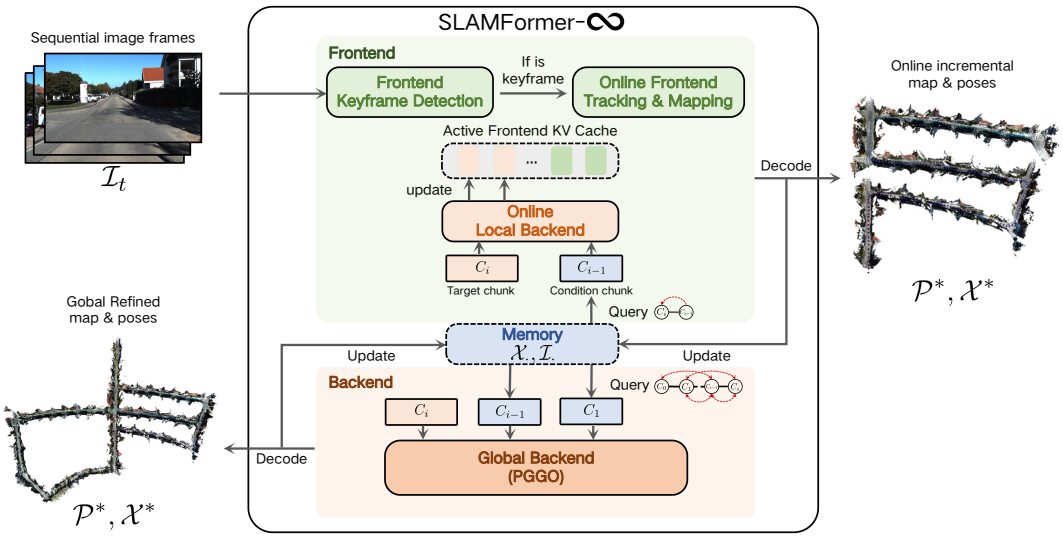

Figure 3: The SLAM pipeline. Frontend detects keyframes and provides online track. After a fixed number of keyframes, we refine the most recent local window to improve short-range pose and geometry consistency. When loop detection is triggered or at sequence end, PGGO is applied with the same transformer function.

3.4 SLAM at Test Time

At test time as in Fig. 3, SLAMFormer-∞ performs streaming reconstruction and pose estimation from RGB sequences $\{\mathbf{I}_t\}_{t=1}^{T}$, with keyframes as $\{\mathbf{I}_n\}_{n=1}^{N}$. We maintain a global graph: $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where each node $\mathbf{x}_n$ stores: $\mathbf{x}_n = (\mathbf{g}_n, \mathbf{P}_n)$.

Frontend. Each incoming frame $\mathbf{I}_i$ is either associated with an existing keyframe or inserted as a new node $\mathbf{I}_n$. Edges are updated based on the local window $\mathcal{W}(n)$. New keyframe $\mathbf{I}_n$ is tracked with eq. (3) for $\mathcal{M}_n$, with the assistance of previous KV caches.

Local Backend. Every c_w keyframes inputs, we trigger one local backend with function eq. (4) for the update of $\hat{\mathcal{M}}_{n-c_w:n}$.

Global Backend. When loop-detection is triggered or after the last frame, given the graph initialization from pose graph or frontend prediction, SLAMFormer-∞ performs PGGO for global refinement: iterating from $k = 1$ to K, for node $\mathbf{x}_n \in \mathcal{V}$, $\tilde{\mathbf{x}}_n^{k+1} = h_\theta(\tilde{\mathbf{x}}_n^k)$. These results yield a globally consistent reconstruction induced by the learned SLAMFormer-∞ prior.

3.5 Training Strategy

As shown in Fig. 4, the model is trained with four shared-weight modes that differ only in attention masks and conditioning patterns, matching test-time frontend, backend, and fine-stage inference.

Training Frontend. Mode 1 trains online tracking and mapping: the first two frames initialize a local coordinate system with full attention, and later frames use causal inter-frame attention.

Training Backend without Memory Condition. Mode 2 trains backend refinement without memory condition. The first half of the clip uses full attention, and the second half is decoded causally from the refined prefix.

Training Backend with Memory Condition. Mode 3 trains backend refinement with memory condition. Using detached Mode-2 predictions, we align a prefix span $[s, N/2)$ to an anchor and encode it as pose-injected frames for the remaining target segment (with s a random integer).

Training Fine Stage with Memory Condition. Mode 4 matches the fine stage by conditioning the middle target chunk on front and back neighboring condition chunks.

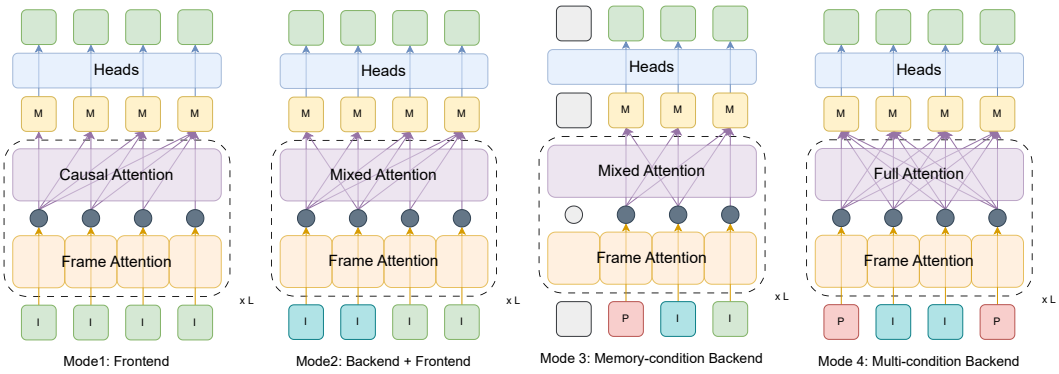

Figure 4: Four training modes of SLAMFormer-∞. I and I represent the image tokens fed into the frontend and backend. P represents the image tokens injected with pose. M represents the map tokens of a frame. and represent the layer intermediate and final output. In each mode, I, I and P are fed into the transformer backbone f_θ, with L layers of frame attention and various inter-frame attentions. Pose and pointmap are regressed by the heads f_ψ.

	Method	LC	Calibration	Recon.	Avg.	Avg.*	00	01	02	03	04	05	06	07	08	09	10
	seq. frames	-	-	-	*2109*	*2210*	*4542*	*1101*	*4661*	*801*	*271*	*2761*	*1101*	*1101*	*4071*	*1591*	*1201*
	seq. length (m)	-	-	-	*2012.243*	*1968.147*	*3724.19*	*2453.20*	*5067.23*	*560.89*	*393.65*	*2205.58*	*1232.88*	*649.70*	*3222.80*	*1705.05*	*919.52*
	seq. speed (m / frame)	-	-	-	*0.95*	*0.89*	*0.82*	*2.23*	*1.09*	*0.70*	*1.45*	*0.80*	*1.12*	*0.59*	*0.79*	*1.07*	*0.77*
	contains loop	-	-	-	-	-	✓	✗	✓	✗	✗	✓	✓	✓	✗	✓	✗
Classic	ORB-SLAM2 [26] (w/o LC)	✗	*Required*	*Sparse*	69.727	26.480	40.65	502.20	47.82	**0.94**	1.30	29.95	40.82	16.04	*43.09*	38.77	**5.42**
Classic	ORB-SLAM2[26] (w/ LC)	✓	*Required*	*Sparse*	54.816	**9.464**	**6.03**	508.34	**14.76**	1.02	1.57	**4.04**	11.16	2.19	38.85	**8.39**	6.63
Classic	LDSO [27]	✓	*Required*	*Sparse*	**22.425**	23.500	9.32	11.68	*31.98*	2.85	1.22	5.10	13.55	2.96	129.02	*21.64*	17.36
Learning Based	DROID-VO [3]	✗	*Required*	*Dense*	54.188	51.187	98.43	84.20	108.80	2.58	0.93	59.27	64.40	24.20	64.55	71.80	16.91
Learning Based	DPVO [28]	✗	*Required*	*Sparse*	53.609	57.701	113.21	12.69	123.40	*2.09*	**0.68**	58.96	54.78	19.26	115.90	75.10	*13.63*
Learning Based	DROID-SLAM [3]	-	*Required*	*Dense*	100.278	75.846	92.10	344.60	107.61	2.38	1.00	118.50	62.47	21.78	161.60	72.32	118.70
Learning Based	DPV-SLAM [29]	✓	*Required*	*Sparse*	53.034	57.187	112.80	**11.50**	123.53	2.50	*0.81*	57.80	54.86	18.77	110.49	76.66	13.65
Learning Based	DPV-SLAM++ [29]	✓	*Required*	*Sparse*	*25.749*	27.138	*8.30*	*11.86*	39.64	2.50	0.78	*5.74*	*11.60*	**1.52**	110.90	76.70	13.70
Learning Based	MASt3R-SLAM [7]	✓	*No Need*	*Dense*	/	/	*TL*	*TL*	*TL*	*TL*	*TL*	*TL*	*TL*	*TL*	*TL*	*TL*	*TL*
Learning Based	CUT3R [30]	✗	*No Need*	*Dense*	/	/	*OOM*	*OOM*	*OOM*	148.07	22.31	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*
Learning Based	Fast3R [12]	✗	*No Need*	*Dense*	/	/	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*
Learning Based	VGGT [13]	✗	*No Need*	*Dense*	/	/	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*
Learning Based	VGGT-Long [19]	✓	*No Need*	*Dense*	26.358	*19.298*	8.06	96.96	34.16	6.83	4.16	9.15	**4.68**	*2.68*	63.15	32.24	27.87
	SLAMFormer-∞	✓	*No Need*	*Dense*	23.011	15.653	15.39	96.58	28.81	4.97	4.16	11.36	13.54	8.53	**37.00**	18.85	13.93

Table 1: KITTI Odometry tracking results. We report ATE RMSE [m] (↓) on sequences 00–10. LC denotes loop closure, Avg.* excludes the high-speed Seq. 01, and OOM/TL denote CUDA out-of-memory on a single RTX 4090/tracking lost. Colors mark **first**, second, and *third* best results.

4 Experiments

4.1 Experimental Setup

Training setup. We initialize two domain-specific SLAMFormer-∞ variants from a pretrained SLAM-Former model and further adapt them to different scene scales and motion statistics. The indoor variant is trained with 12-frame clips at long side 518, and the outdoor variant with 36-frame clips at long side 224. Both variants are trained for 10 epochs on 48 A100 GPUs. See Appendix A for the dataset composition and optimization hyperparameters.

Evaluation protocol. We evaluate SLAMFormer-∞ on indoor benchmarks (Replica [21], TUM RGB-D [22], and 7-Scenes [23]) and outdoor driving benchmarks (KITTI Odometry [24] and Waymo Open Dataset [25]). Tracking accuracy is reported as ATE RMSE in meters, and dense reconstruction is evaluated using pointmap accuracy, completeness, and Chamfer distance. All reported metrics are lower-is-better. Calibration-free methods are evaluated without camera intrinsics, while calibrated baselines use their standard calibrated inputs.

4.2 Large Outdoor Scenes

On outdoor sequences, the pose initialization for the iterative backend PGGO (fine stage) is obtained from a pose-graph optimization configured consistently with VGGT-Long [19].

Tracking performance. Table 1 and Table 2 report tracking results on KITTI and Waymo, respectively. On KITTI, SLAMFormer-∞ improves over VGGT-Long from 26.358m to 23.011m

Segment ID	Calib.	Avg.	163453191	183829460	315615587	346181117	371159869	405841035	460417311	520018670	610454533
Frame num.	-	*198*	*198*	*199*	*199*	*199*	*196*	*199*	*198*	*199*	*198*
Segment length	-	*172.533*	*159.963*	*42.301*	*165.149*	*351.213*	*272.661*	*85.743*	*265.906*	*134.552*	*62.739*
Segment speed	-	*0.871*	*0.808*	*0.213*	*0.830*	*1.765*	*1.391*	*0.431*	*1.343*	*0.676*	*0.317*
Traffic	-	-	*Low*	*High*	*Low*	*Low*	*Medium*	*Low*	*Medium*	*Low*	*High*
DROID-SLAM [3]	*Required*	*4.396*	*3.705*	**0.301**	**0.447**	*8.653*	9.320	7.621	*4.170*	*TL*	**0.264**
MASt3R-SLAM [7]	*No Need*	5.560	4.500	0.556	1.833	12.544	*8.601*	1.412	5.428	*7.910*	1.195
CUT3R [30]	*No Need*	9.872	8.781	3.810	5.790	24.015	13.070	7.261	13.206	8.597	3.229
Fast3R [12]	*No Need*	/	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*
VGGT [13]	*No Need*	/	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*
VGGT-Long [19]	*No Need*	1.996	1.753	2.629	0.559	3.452	**3.343**	*1.444*	**1.541**	**2.547**	*0.455*
SLAMFormer-∞	*No Need*	**1.813**	**1.270**	*0.616*	*0.810*	**1.464**	5.281	**0.568**	3.116	2.825	0.371

Table 2: Waymo tracking results. We report ATE RMSE [m] (↓) on nine urban driving segments. Gray rows provide segment metadata, and OOM/TL denote CUDA out-of-memory/tracking lost. Colors mark **first**, second, and *third* best results.

Segment ID	Metric	Calib.	Avg.	163453191	183829460	315615587	346181117	371159869	405841035	460417311	520018670	610454533
Frame num.	-	-	*198*	*198*	*199*	*199*	*199*	*196*	*199*	*198*	*199*	*198*
Segment length	-	-	*172.533*	*159.963*	*42.301*	*165.149*	*351.213*	*272.661*	*85.743*	*265.906*	*134.552*	*62.739*
Segment speed	-	-	*0.871*	*0.808*	*0.213*	*0.830*	*1.765*	*1.391*	*0.431*	*1.343*	*0.676*	*0.317*
Traffic	-	-	-	*Low*	*High*	*Low*	*Low*	*Medium*	*Low*	*Medium*	*Low*	*High*
DROID-SLAM [3]	Accuracy ↓	*Required*	*1.201*	0.781	*1.136*	*2.247*	*2.393*	**1.090**	0.539	**0.740**	*TL*	0.677
	Completeness ↓		*8.540*	4.610	10.245	5.540	8.669	8.592	11.144	5.320	*TL*	14.201
	Chamfer ↓		*4.870*	2.696	5.691	3.893	5.531	4.841	5.842	*3.030*	*TL*	7.439
MASt3R-SLAM [7]	Accuracy ↓	*No Need*	3.772	3.189	2.988	3.787	4.689	4.436	1.166	4.637	*6.417*	2.637
	Completeness ↓		*3.177*	**1.715**	3.284	*2.047*	2.981	**2.679**	**2.895**	2.002	*4.429*	*6.560*
	Chamfer ↓		*3.474*	*2.452*	*3.136*	*2.917*	*3.835*	*3.558*	2.031	3.319	*5.423*	*4.599*
CUT3R [30]	Accuracy ↓	*No Need*	3.884	3.580	1.144	2.418	3.712	3.679	4.346	2.012	12.320	1.744
	Completeness ↓		6.801	8.251	9.352	8.748	8.537	5.467	*3.393*	6.164	2.302	8.999
	Chamfer ↓		5.343	5.916	5.248	5.583	6.125	4.573	3.869	4.088	7.311	5.371
VGGT-Long [19]	Accuracy ↓	*No Need*	1.182	*1.002*	**0.395**	**0.925**	1.668	*2.580*	*0.679*	0.784	1.358	*1.246*
	Completeness ↓		2.860	*2.762*	*3.417*	**1.738**	*3.261*	2.791	3.216	**1.840**	4.694	**2.022**
	Chamfer ↓		2.021	1.882	1.906	**1.331**	2.465	2.685	**1.948**	**1.312**	3.026	1.634
SLAMFormer-∞	Accuracy ↓	*No Need*	**0.949**	**0.601**	0.467	1.454	**1.064**	1.964	**0.251**	*1.268*	**0.935**	**0.540**
	Completeness ↓		**2.777**	2.562	**3.191**	1.965	**2.815**	*3.259*	5.461	*2.174*	**1.025**	2.538
	Chamfer ↓		**1.863**	**1.582**	**1.829**	1.709	**1.939**	**2.611**	*2.856*	1.721	**0.980**	**1.539**

Table 3: Waymo point-map reconstruction results. We report accuracy, completeness, and Chamfer distance (↓) against LiDAR point clouds; lower is better for all metrics. Gray rows provide segment metadata, and TL denotes tracking lost. Colors mark **first**, second, and *third* best results.

average ATE RMSE on full sequences, and on Waymo it further reduces the average ATE RMSE from 1.996m to 1.813m across urban segments with diverse speeds, lengths, and traffic densities. Taken together, these results indicate that the memory-conditioned frontend and backend improve tracking on large outdoor sequences by providing stronger long-range pose guidance and refinement than global pose alignment alone.

Reconstruction performance. Table 3 evaluates dense point-map reconstruction on Waymo. Beyond trajectory accuracy, SLAMFormer-∞ also improves the average dense geometry over VGGT-Long: accuracy decreases from 1.182 to 0.949, completeness from 2.860 to 2.777, and Chamfer distance from 2.021 to 1.863.

Please also find in Fig. 1 the qualitative comparison on KITTI 05 and 09 sequences, where VGGT-Long's reconstructions are highly mismatched, while ours demonstrate smooth reconstructions. Please find the Appendix B for more demonstrations.

4.3 Small Indoor Scenes

For indoor evaluation, we follow the VGGT-SLAM [8] protocol by using a one-frame overlap between adjacent windows for error estimation and boundary-error correction. Indoor benchmarks evaluate whether the pose condition design preserves local accuracy under short trajectories, narrow baselines, and frequent viewpoint changes. Table 4 reports aggregate tracking and reconstruction results on TUM RGB-D, 7-Scenes, and Replica. SLAMFormer-∞ remains comparable to state-of-the-art indoor SLAM systems while preserving calibration-free dense reconstruction. On 7-Scenes, it improves over VGGT-SLAM [8] in both tracking and geometry: ATE RMSE decreases from 0.068m to 0.046m, and accuracy/completeness/Chamfer improve from $0.054/0.060/0.057$ to $0.029/0.049/0.039$. Similar trends hold on TUM RGB-D and Replica, where our method achieves competitive tracking accuracy and high-quality reconstruction. In particular, matching-driven methods like MASt3R-SLAM [7] and EC3R-SLAM [31] excel on Replica due to near-perfect matching on noise-free simulated images, but degrade on real-world datasets. SLAM-Former [9] achieves

Method	TUM RGB-D	7-Scenes				Replica		
	ATE↓	ATE↓	Acc.↓	Comp.↓	Chamf.↓	ATE↓	Acc.↓	Comp.↓
CUT3R [30]	0.113	0.073	0.032	0.047	0.040	0.170	7.52	3.62
StreamVGGT [32]	0.187	0.081	0.058	0.057	0.057	0.125	9.88	4.73
VGGT-SLAM [8]	0.084	0.068	0.054	0.060	0.057	0.071	7.52	5.86
MASt3R-SLAM [7]	0.061	0.065	0.065	0.067	0.056	0.045	2.92	2.25
EC3R-SLAM [31]	0.070	0.075	0.025	0.054	0.040	0.041	2.82	2.07
VGGT-Long [19]$^{+}$	0.082	0.089	0.049	0.081	0.065	0.071	5.80	3.17
SLAM-Former [9]	**0.039**	**0.042**	**0.017**	**0.037**	**0.027**	**0.030**	**2.09**	**1.56**
SLAMFormer-∞	0.066	0.046	0.029	0.049	0.039	0.052	6.00	3.33
SLAMFormer-∞ (w/o fine)	0.068	0.047	0.030	0.049	0.040	0.061	6.11	3.39

Table 4: Indoor tracking and reconstruction results on TUM RGB-D, 7-Scenes, and Replica. We report ATE RMSE [m] for tracking, and accuracy, completeness, and Chamfer distance for reconstruction. Lower is better for all metrics. $^{+}$ indicates results from our own run.

Figure 5: Qualitative effect of the fine stage (PGGO) on Replica. Compared with the coarse prediction, the fine-stage output produces cleaner local surfaces and more stable alignment.

the overall best performance on indoor benchmarks, benefiting from its fully end-to-end learned architecture. In contrast, SLAMFormer-∞ is designed primarily for global optimization over much longer trajectories, and its global optimization is not learned end-to-end. Therefore, on short indoor benchmarks, ours may still underperform fully data-driven models that are tailored specifically to the same distributions, whereas ours prioritizes global optimization for unbounded ones.

4.4 Ablation Study: Before and After Fine Stage (PGGO)

Table 4 also compares SLAMFormer-∞ with and without the fine stage. Quantitatively, the fine stage brings consistent gains across the indoor benchmarks. The clearest improvement appears on Replica, where ATE RMSE decreases from 0.061m to 0.052m and reconstruction accuracy/completeness improve from $6.11/3.39$ to $6.00/3.33$. TUM RGB-D and 7-Scenes show the same overall trend, indicating that the fine stage improves local geometric consistency, although the numerical margins remain limited. By contrast, the qualitative effect is more evident: Fig. 5 shows cleaner point-map surfaces and reduced local drift after the fine stage, making the visual improvement more noticeable than the score differences alone suggest.

5 Limitations

Unlike SLAM-Former that implicitly construct the frame-connections, SLAMFormer-∞'s PGGO inputs a pre-defined graph, from frontend and loop-detection. The quality of graph-connectivity has effect to the performance and is not learned from data.

6 Conclusion

We propose SLAMFormer-∞ to address the SLAM Transformer's limitation that cannot learn to tackle unbounded-long distance sequences. Based on our memory-condition design, SLAMFormer-∞ achieved efficient online dense reconstruction in frontend and joint trajectory-geometry optimization in the backend for unbounded-long sequences. Our method has demonstrated better performance across large-scale datasets and has exhibited generalization to extremely long trajectories exceeding 17km.

References

[1] A. J. Davison, I. D. Reid, N. D. Molton, and O. Stasse. Monoslam: Real-time single camera slam. *IEEE TPAMI*, 2007.

[2] G. Klein and D. Murray. Parallel tracking and mapping for small ar workspaces. In *2007 6th IEEE and ACM international symposium on mixed and augmented reality*. IEEE, 2007.

[3] Z. Teed and J. Deng. Droid-slam: Deep visual slam for monocular, stereo, and rgb-d cameras. In *NeurIPS*, 2021.

[4] B. Ummenhofer, H. Zhou, J. Uhrig, N. Mayer, E. Ilg, A. Dosovitskiy, and T. Brox. Demon: Depth and motion network for learning monocular stereo. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 5038–5047, 2017.

[5] H. Matsuki, R. Murai, P. H. Kelly, and A. J. Davison. Gaussian splatting slam. In *CVPR*, 2024.

[6] W. Zhang, Q. Cheng, D. Skuddis, N. Zeller, D. Cremers, and N. Haala. Hi-slam2: Geometry-aware gaussian slam for fast monocular scene reconstruction. *IEEE Transactions on Robotics*, 2025.

[7] R. Murai, E. Dexheimer, and A. J. Davison. Mast3r-slam: Real-time dense slam with 3d reconstruction priors. In *CVPR*, 2025.

[8] D. Maggio, H. Lim, and L. Carlone. Vggt-slam: Dense rgb slam optimized on the sl (4) manifold. In *NeurIPS*, 2026.

[9] Y. Yuan, Z. Chen, K. Li, W. Wang, and H. Zhao. Slam-former: Putting slam into one transformer. *arXiv preprint arXiv:2509.16909*, 2025.

[10] S. Wang, V. Leroy, Y. Cabon, B. Chidlovskii, and J. Revaud. Dust3r: Geometric 3d vision made easy. In *CVPR*, 2024.

[11] V. Leroy, Y. Cabon, and J. Revaud. Grounding image matching in 3d with mast3r. In *ECCV*, 2024.

[12] J. Yang, A. Sax, K. J. Liang, M. Henaff, H. Tang, A. Cao, J. Chai, F. Meier, and M. Feiszli. Fast3r: Towards 3d reconstruction of 1000+ images in one forward pass. In *CVPR*, 2025.

[13] J. Wang, M. Chen, N. Karaev, A. Vedaldi, C. Rupprecht, and D. Novotny. Vggt: Visual geometry grounded transformer. In *CVPR*, 2025.

[14] Y. Wang, J. Zhou, H. Zhu, W. Chang, Y. Zhou, Z. Li, J. Chen, J. Pang, C. Shen, and T. He. π^3: Permutation-equivariant visual geometry learning. In *ICLR*, 2026.

[15] X. Chen, Y. Chen, Y. Xiu, A. Geiger, and A. Chen. Ttt3r: 3d reconstruction as test-time training. In *ICLR*, 2026.

[16] C. Cheng, X. Chen, T. Xie, W. Yin, W. Ren, Q. Zhang, X. Guo, and H. Wang. Longstream: Long-sequence streaming autoregressive visual geometry. In *CVPR*, 2026.

[17] L.-Z. Chen, J. Gao, Y. Chen, K. L. Cheng, Y. Sun, L. Hu, N. Xue, X. Zhu, Y. Shen, Y. Yao, et al. Geometric context transformer for streaming 3d reconstruction. *arXiv preprint arXiv:2604.14141*, 2026.

[18] Y. Yuan, M. Bleier, and A. Nüchter. Scenefactory: A workflow-centric and unified framework for incremental scene modeling. *IEEE Transactions on Robotics*, 41:3183–3201, 2025.

[19] K. Deng, Z. Ti, J. Xu, J. Yang, and J. Xie. Vggt-long: Chunk it, loop it, align it–pushing vggt's limits on kilometer-scale long rgb sequences. In *ICRA*, 2026.

[20] Y. Liu, S. Dong, S. Wang, Y. Yin, Y. Yang, Q. Fan, and B. Chen. Slam3r: Real-time dense scene reconstruction from monocular rgb videos. In *CVPR*, 2025.

[21] E. Sucar, S. Liu, J. Ortiz, and A. J. Davison. imap: Implicit mapping and positioning in real-time. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 6229–6238, 2021.

[22] J. Sturm, N. Engelhard, F. Endres, W. Burgard, and D. Cremers. A benchmark for the evaluation of rgb-d slam systems. In *2012 IEEE/RSJ international conference on intelligent robots and systems*, pages 573–580. IEEE, 2012.

[23] B. Glocker, S. Izadi, J. Shotton, and A. Criminisi. Real-time rgb-d camera relocalization. In *2013 IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, pages 173–179. IEEE, 2013.

[24] A. Geiger, P. Lenz, and R. Urtasun. Are we ready for autonomous driving? the kitti vision benchmark suite. In *2012 IEEE conference on computer vision and pattern recognition*, pages 3354–3361. IEEE, 2012.

[25] A. Gaidon, Q. Wang, Y. Cabon, and E. Vig. Virtual worlds as proxy for multi-object tracking analysis. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 4340–4349, 2016.

[26] R. Mur-Artal and J. D. Tardós. Orb-slam2: An open-source slam system for monocular, stereo, and rgb-d cameras. *IEEE transactions on robotics*, 2017.

[27] X. Gao, R. Wang, N. Demmel, and D. Cremers. Ldso: Direct sparse odometry with loop closure. In *IROS*. IEEE, 2018.

[28] Z. Teed, L. Lipson, and J. Deng. Deep patch visual odometry. In *NeurIPS*, 2023.

[29] L. Lipson, Z. Teed, and J. Deng. Deep patch visual slam. In *ECCV*, 2024.

[30] Q. Wang, Y. Zhang, A. Holynski, A. A. Efros, and A. Kanazawa. Continuous 3d perception model with persistent state. In *CVPR*, 2025.

[31] L. Hu, N. A. Oufroukh, F. Bonardi, and R. Ghandour. Ec3r-slam: Efficient and consistent monocular dense slam with feed-forward 3d reconstruction. *arXiv preprint arXiv:2510.02080*, 2025.

[32] D. Zhuo, W. Zheng, J. Guo, Y. Wu, J. Zhou, and J. Lu. Streaming 4d visual geometry transformer. In *ICLR*, 2026.

[33] G. Baruch, Z. Chen, A. Dehghan, T. Dimry, Y. Feigin, P. Fu, T. Gebauer, B. Joffe, D. Kurz, A. Schwartz, et al. Arkitscenes: A diverse real-world dataset for 3d indoor scene understanding using mobile rgb-d data. *arXiv preprint arXiv:2111.08897*, 2021.

[34] C. Yeshwanth, Y.-C. Liu, M. Nießner, and A. Dai. Scannet++: A high-fidelity dataset of 3d indoor scenes. In *ICCV*, 2023.

[35] A. Dai, A. X. Chang, M. Savva, M. Halber, T. Funkhouser, and M. Nießner. Scannet: Richly-annotated 3d reconstructions of indoor scenes. In *CVPR*, 2017.

[36] M. Roberts, J. Ramapuram, A. Ranjan, A. Kumar, M. A. Bautista, N. Paczan, R. Webb, and J. M. Susskind. Hypersim: A photorealistic synthetic dataset for holistic indoor scene understanding. In *ICCV*, 2021.

[37] Y. Yao, Z. Luo, S. Li, J. Zhang, Y. Ren, L. Zhou, T. Fang, and L. Quan. Blendedmvs: A large-scale dataset for generalized multi-view stereo networks. In *CVPR*, 2020.

[38] Z. Li and N. Snavely. Megadepth: Learning single-view depth prediction from internet photos. In *CVPR*, 2018.

[39] Y. Cabon, N. Murray, and M. Humenberger. Virtual kitti 2. *arXiv preprint arXiv:2001.10773*, 2020.

[40] M. Patel, F. Yang, Y. Qiu, C. Cadena, S. Scherer, M. Hutter, and W. Wang. Tartanground: A large-scale dataset for ground robot perception and navigation. In *IROS*, 2025.

[41] K. Yadav, R. Ramrakhya, S. K. Ramakrishnan, T. Gervet, J. Turner, A. Gokaslan, N. Maestre, A. X. Chang, D. Batra, M. Savva, et al. Habitat-matterport 3d semantics dataset. In *CVPR*, 2023.

A Training Details

Table 5: Dataset composition of the indoor and outdoor training configurations.

Configuration	Datasets	Clip length	Long side
Indoor	ARKitScenes [33], ScanNet++ [34], ScanNet [35], HyperSim [36], BlendedMVS [37], MegaDepth [38]	12	518
Outdoor	VirtualKITTI2 [39], TartanGround [40], HabitatHM3D [41], ARKitScenes [33], ScanNet++ [34], BlendedMVS [37], MegaDepth [38]	36	224

Table 6: Optimization hyperparameters used for both training configurations.

Hyperparameter	Value
Batch size	1 per GPU
Gradient accumulation	None
Optimizer	AdamW
Weight decay	0.05
Initial learning rate	1×10^{-5}
Minimum learning rate	1×10^{-8}
LR schedule	Cosine decay
Warm-up	0.5 epochs
Precision	Mixed precision
Training epochs	10
GPUs	48 A100
Indoor training time	$\sim$1 hour / epoch
Outdoor training time	$\sim$2.5 hours / epoch

B Qualitative Analysis on KITTI

Fig. 6 provides additional qualitative comparisons on KITTI Odometry sequences. VGGT-Long achieves global pose alignment and preserves the coarse trajectory layout, but its dense geometry is mainly stitched after pose correction. As a result, local structures often remain fragmented or misaligned, as shown by the zoomed regions where nearby surfaces become discontinuous and poorly fused.

In contrast, SLAMFormer-∞ performs joint pose-geometry refinement through the memory-anchored backend. The optimized poses and pointmaps are updated together, producing more coherent local geometry while maintaining large-scale trajectory consistency over hundreds of meters to kilometer-scale routes. These results show that the proposed backend improves reconstruction quality beyond trajectory-level alignment, especially in long outdoor sequences where local geometric errors can accumulate after pose-only optimization.

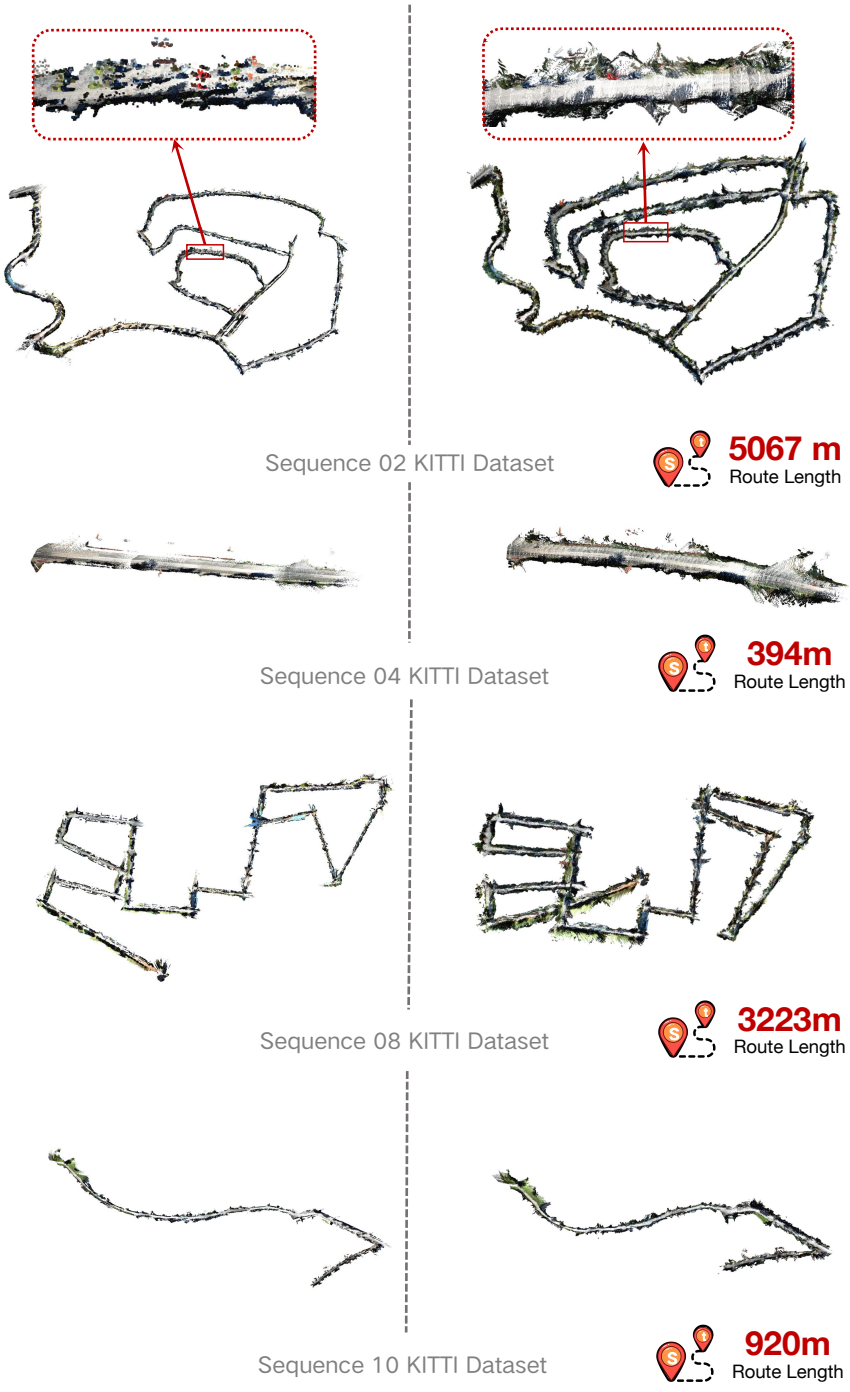

Figure 6: Additional qualitative comparisons on KITTI Odometry. VGGT-Long performs global pose alignment but leaves local geometry largely unrefined, whereas SLAMFormer-∞ jointly refines pose and dense geometry, producing more coherent large-scale reconstructions.